University Of Engineering and Technology, Lahore
Computer Engineering Department

Lab Evaluation:

Course Name: Object Oriented Programming	**Course Code:** CMPE 221L
Assignment Type: Lab	**Dated:**
Semester: 2nd	**Session:** CYS 2025 Spring
Lab/Project/Assignment #: 4	**CLOs to be covered:** CLO1
Lab Title: Classes & Objects	**Teacher Name:** Hafiz Muhammad Mubashar Imran

CLO1	Implement abstraction and encapsulation to develop reusable classes for objects of real-world problems					
Levels (Marks)	**Level1**	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(10)						
					Total	**/10**

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	**9-10**	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	**7-8**	L2	Submitted the lab tasks but have good understanding
Good	**5-6**	L3	Submitted the lab tasks but have weak understanding.
Basic	**3-4**	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	**1-2**	L5	Submitted only one lab task.
Not Acceptable	**0**	L6	Did not attempt

LAB No. 4

Lab Goals/Objectives:

- Basic OO programming in java
- Objects and Classes

Equipment Required: Inteli J IDEA Edition Community, Net Beans with JDK 13 Version, Visual Studio.
Theory/Relevant Material: Objects and classes, constructors, getter & setters, modifiers

1. Thinking of Time as an entity JTime class was created. Each object of this class represents a time-stamp (a mark in history). It has three attributes (hour, minute, second) and three constructors along with some other functions. Enhance its functionality by added following functions to it and test the functionality by updating the driver function.
a. Add a function that finds the time elapsed in seconds between two time-stamps.
b. Add a function that finds the time elapsed in two time-stamps in format of another time-stamp.
c. Add another constructor that takes total number of seconds elapsed after 12 am and construct the timestamp accordingly.

Code:

```
public class Task_1 {
    public static void main(String[] args) {
        JTime t1 = new JTime(8, 30, 15);
        JTime t2 = new JTime(14, 45, 50);
        JTime midnight = new JTime();
        JTime copyOfT1 = new JTime(t1);
        System.out.println("t1       = " + t1);
        System.out.println("t2       = " + t2);
        System.out.println("midnight = " + midnight);
        System.out.println("copy of t1 = " + copyOfT1);
        System.out.println();
        System.out.println("--- a) elapsed time in seconds ---");
        System.out.println("Between t1 and t2: " + t1.secondsBetween(t2) + " seconds");
        System.out.println("Between t2 and t1: " + t2.secondsBetween(t1) + " seconds (same,
order does not matter)");
        System.out.println();
        System.out.println("--- b) elapsed time as another time-stamp ---");
        JTime gap = t1.timeBetween(t2);
        System.out.println("Between t1 and t2: " + gap + "  (" + gap.getHour() + " h "
                + gap.getMinute() + " m " + gap.getSecond() + " s)");
        System.out.println();
        System.out.println("--- c) constructor from seconds after 12 am ---");
        JTime fromSeconds1 = new JTime(3725);
        JTime fromSeconds2 = new JTime(45296);
        JTime fromSeconds3 = new JTime(86399);
        JTime fromSeconds4 = new JTime(86405);
        System.out.println("3725 seconds   -> " + fromSeconds1);
        System.out.println("45296 seconds  -> " + fromSeconds2);
        System.out.println("86399 seconds  -> " + fromSeconds3);
        System.out.println("86405 seconds  -> " + fromSeconds4 + "  (wrapped past midnight)");
        System.out.println();
        System.out.println("--- extra: invalid values are rejected ---");
        JTime bad = new JTime(25, 61, 70);
        System.out.println("bad = " + bad);
    }
}
class JTime {
    private int hour;
    private int minute;
    private int second;
    public JTime() {
        hour = 0;
        minute = 0;
        second = 0;
```

```java
    }
    public JTime(int hour, int minute, int second) {
        setHour(hour);
        setMinute(minute);
        setSecond(second);
    }
    public JTime(JTime other) {
        hour = other.hour;
        minute = other.minute;
        second = other.second;
    }
    public JTime(int totalSeconds) {
        if (totalSeconds < 0) {
            System.out.println("Invalid number of seconds: " + totalSeconds + " (using 00:00:00)");
            return;
        }
        totalSeconds = totalSeconds % 86400;
        hour = totalSeconds / 3600;
        minute = (totalSeconds % 3600) / 60;
        second = totalSeconds % 60;
    }
    public int getHour()   { return hour; }
    public int getMinute() { return minute; }
    public int getSecond() { return second; }
    public void setHour(int hour) {
        if (hour >= 0 && hour <= 23) this.hour = hour;
        else System.out.println("Invalid hour: " + hour + " (must be 0-23)");
    }
    public void setMinute(int minute) {
        if (minute >= 0 && minute <= 59) this.minute = minute;
        else System.out.println("Invalid minute: " + minute + " (must be 0-59)");
    }
    public void setSecond(int second) {
        if (second >= 0 && second <= 59) this.second = second;
        else System.out.println("Invalid second: " + second + " (must be 0-59)");
    }
    public int toSeconds() {
        return hour * 3600 + minute * 60 + second;
    }
    public int secondsBetween(JTime other) {
        int difference = this.toSeconds() - other.toSeconds();
        if (difference < 0) difference = -difference;
        return difference;
    }
    public JTime timeBetween(JTime other) {
        return new JTime(secondsBetween(other));
    }
    public String toString() {
        return String.format("%02d:%02d:%02d", hour, minute, second);
    }
}
```

Output:

```
t1       = 08:30:15
t2       = 14:45:50
midnight = 00:00:00
copy of t1 = 08:30:15

--- a) elapsed time in seconds ---
Between t1 and t2: 22535 seconds
Between t2 and t1: 22535 seconds (same, order does not matter)

--- b) elapsed time as another time-stamp ---
Between t1 and t2: 06:15:35  (6 h 15 m 35 s)

--- c) constructor from seconds after 12 am ---
3725 seconds   -> 01:02:05
45296 seconds  -> 12:34:56
86399 seconds  -> 23:59:59
86405 seconds  -> 00:00:05  (wrapped past midnight)

--- extra: invalid values are rejected ---
Invalid hour: 25 (must be 0-23)
Invalid minute: 61 (must be 0-59)
Invalid second: 70 (must be 0-59)
bad = 00:00:00
```

2. We are going to develop a menu-driven application to manage a simple bank account class. It supports the following operations
a. Deposit money (to add an amount in current balance)
b. Withdraw money (to subtract an amount from the current balance)
c. Check balance (to display current balance of customer)
Note: Use a Switch statement to enter the choice of operation from the user. For initially create a function withname saving account to store the current balance of the customer. Write a Java class along with a driver program to test it.

Code:

```java
import java.util.Scanner;
public class Task_2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Customer name: ");
        String name = sc.nextLine();
        double opening;
        while (true) {
```

```
            System.out.print("Opening balance: ");
            opening = readAmount(sc);
            if (opening >= 0) break;
            System.out.println("Please enter a valid number (0 or more).");
        }
        SavingAccount account = new SavingAccount(name, opening);
        boolean running = true;
        while (running) {
            System.out.println();
            System.out.println("===== SAVING ACCOUNT MENU =====");
            System.out.println("1. Deposit money");
            System.out.println("2. Withdraw money");
            System.out.println("3. Check balance");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
            if (!sc.hasNextLine()) break;
            int choice;
            try {
                choice = Integer.parseInt(sc.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.println("Please type a number from 1 to 4.");
                continue;
            }
            switch (choice) {
                case 1:
                    System.out.print("Amount to deposit: ");
                    account.deposit(readAmount(sc));
                    break;
                case 2:
                    System.out.print("Amount to withdraw: ");
                    account.withdraw(readAmount(sc));
                    break;
                case 3:
                    account.checkBalance();
                    break;
                case 4:
                    System.out.println("Thank you, " + account.getName() + ". Goodbye!");
                    running = false;
                    break;
                default:
                    System.out.println("Invalid choice. Please pick 1, 2, 3 or 4.");
            }
        }
    }
    static double readAmount(Scanner sc) {
        if (!sc.hasNextLine()) return -1;
        try {
            double v = Double.parseDouble(sc.nextLine().trim());
            if (Double.isNaN(v) || Double.isInfinite(v)) return -1;
            return v;
        } catch (NumberFormatException e) {
            return -1;
        }
    }
}
class SavingAccount {
    private String name;
    private double balance;
    public SavingAccount(String name, double initialBalance) {
        this.name = name;
        if (initialBalance >= 0) {
            balance = initialBalance;
```

```
        } else {
            System.out.println("Opening balance cannot be negative. Starting with 0.");
            balance = 0;
        }
    }
    public String getName() { return name; }
    public void deposit(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid amount. Deposit must be more than 0.");
            return;
        }
        balance = balance + amount;
        System.out.println("Deposited " + amount + ". New balance: " + balance);
    }
    public void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid amount. Withdrawal must be more than 0.");
        } else if (amount > balance) {
            System.out.println("Not enough money. Your balance is only " + balance);
        } else {
            balance = balance - amount;
            System.out.println("Withdrew " + amount + ". New balance: " + balance);
        }
    }
    public void checkBalance() {
        System.out.println(name + ", your current balance is: " + balance);
    }
}
```

Output:

```
Customer name: anaya
Opening balance: 5000
```

```
===== SAVING ACCOUNT MENU =====
1. Deposit money
2. Withdraw money
3. Check balance
4. Exit
Enter your choice: 1
Amount to deposit: 4000
Deposited 4000.0. New balance: 9000.0

===== SAVING ACCOUNT MENU =====
1. Deposit money
2. Withdraw money
3. Check balance
4. Exit
Enter your choice: 2
Amount to withdraw: 3000
Withdrew 3000.0. New balance: 6000.0
```

```
===== SAVING ACCOUNT MENU =====
1. Deposit money
2. Withdraw money
3. Check balance
4. Exit
Enter your choice: 3
anaya, your current balance is: 6000.0
```

Something is Brewing:
3. So lately Ali has been walking as per your instructions. Avoiding obstacles in its path. Now Hasan wants to
startwalking with Ali. But Hasan will walk randomly and will avoid the same obstacles as Ali. Write a code
that will simulate both the walk patterns of Ali and Hasan simultaneously. Output can e of the form:
Starting: Ali's Location is = (0 : 0)
Hassan's Location is = (0 : 0) Hasan has Random Movement, Ali follows order
ALi: Where should i go now: v
Hasan is at = (0 : 1)Ali is at = (0 : -1)
ALi: Where should i go now: ^
Hasan is at = (0 : 2)Ali is at = (0 : 0)
ALi: Where should i go now: ^ Ali is at = (0 : 1)
Hasan encounters hurdle at (1:2) Hasan is at = (0 : 2)

Code:

```java
import java.util.Random;
import java.util.Scanner;
public class Task_3 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random random = new Random();
        Field field = new Field();
        Walker ali = new Walker("Ali");
        Walker hasan = new Walker("Hasan");
        System.out.println("Hurdles are at: " + field.hurdleList());
        System.out.println("Starting: Ali's Location is = " + ali.where()
                + "  Hasan's Location is = " + hasan.where());
        System.out.println("Hasan has random movement, Ali follows orders (^ v < >, q to quit)");
        while (true) {
            System.out.print("\nAli: Where should I go now: ");
            if (!sc.hasNextLine()) break;
            String order = sc.nextLine().trim();
            if (order.equals("q")) {
                System.out.println("Simulation finished.");
                break;
            }
            if (!(order.equals("^") || order.equals("v") || order.equals("<") || order.equals(">"))) {
                System.out.println("Please type ^ v < > or q");
                continue;
```

```
            }
            // Ali's turn
            int dx = 0, dy = 0;
            if (order.equals("^")) dy = 1;
            if (order.equals("v")) dy = -1;
            if (order.equals("<")) dx = -1;
            if (order.equals(">")) dx = 1;
            ali.move(dx, dy, field);
            // Hasan's turn
            int direction = random.nextInt(4);
            dx = 0;
            dy = 0;
            if (direction == 0) dy = 1;
            if (direction == 1) dy = -1;
            if (direction == 2) dx = -1;
            if (direction == 3) dx = 1;
            hasan.move(dx, dy, field);
        }
    }
}
class Field {
    private int[][] hurdles = { {1, 2}, {-1, 1}, {2, -1}, {0, -3}, {-2, -2} };

    public boolean isHurdle(int x, int y) {
        for (int i = 0; i < hurdles.length; i++) {
            if (hurdles[i][0] == x && hurdles[i][1] == y) return true;
        }
        return false;
    }
    public String hurdleList() {
        String s = "";
        for (int i = 0; i < hurdles.length; i++) {
            s = s + "(" + hurdles[i][0] + " : " + hurdles[i][1] + ") ";
        }
        return s;
    }
}
class Walker {
    private String name;
    private int x = 0;
    private int y = 0;
    public Walker(String name) { this.name = name; }
    public String where() { return "(" + x + " : " + y + ")"; }
    public void move(int dx, int dy, Field field) {
        int newX = x + dx;
        int newY = y + dy;
        if (field.isHurdle(newX, newY)) {
            System.out.println(name + " encounters hurdle at (" + newX + ":" + newY + ")  "
                    + name + " is at = " + where());
        } else {
            x = newX;
            y = newY;
            System.out.println(name + " is at = " + where());
        }
    }
}
```

Output:

```
Hurdles are at: (1 : 2) (-1 : 1) (2 : -1) (0 : -3) (-2 : -2)
Starting: Ali's Location is = (0 : 0)  Hasan's Location is = (0 : 0)
Hasan has random movement, Ali follows orders (^ v < >, q to quit)
```

```
Ali: Where should I go now: ^
  Ali: Where should I go now: >
  Ali is at = (1 : 1)
  Hasan is at = (1 : 1)
```

```
Ali: Where should I go now: <
Ali is at = (0 : 1)
Hasan is at = (1 : 0)
```

```
Ali: Where should I go now: <
Ali encounters hurdle at (-1:1)  Ali is at = (0 : 1)
Hasan is at = (0 : 0)
```